

Project. Creating a multi-object tracker

Fernando Fernández del Cerro

I. INTRODUCTION

This project focuses on the study and improvement of a deep learning model for multiple object tracking. It starts from a baseline model consisting of an *object detector* with a *Fast-RCNN* architecture, trained on the labeled dataset **MOT16**, and an object tracker (`tracker`) that assigns current detections to previous ones using a greedy Intersection over Union (*IoU*) strategy. In this way, the achieved *MOTA* score is improved by optimizing certain aspects of the baseline model.

Regarding the mandatory tasks, these include completing and understanding the provided tutorial to grasp the operation of the baseline model, studying the scripts `tracker.py` and `object_detector.py`, and finally, designing, implementing, and evaluating the proposed improvements.

As for the considered improvements, the goal is to take into account only high-quality detections from the detector, by selecting (*tuning*) the optimal threshold that maximizes the F1-score in object detection, as well as applying the *Hungarian algorithm* [4] for data association – a combination that leads to a very significant improvement in the metrics used to evaluate the model.

II. METHOD

The baseline model defines an object detector, which consists of a class called `FRCNN_FPN`, inherited from `FasterRCNN` of the `torchvision` library. It is composed of the following attributes:

- `GeneralizedRCNNTransform`: Module responsible for preprocessing the input images. It performs normalization and dimensionality adjustment operations on the input data.
- `BackBoneWithFPN`: Backbone network architecture for extracting features from the input image. In this case, a *CNN* followed by an *FPN* is used.
- `IntermediateLayerGetter`: Module that extracts feature maps from different layers, allowing access to them.
- `Bottleneck`: This module is responsible for reducing dimensions and increasing the number of channels in the feature maps. These blocks consist of a series of convolutional layers with batch normalization and *ReLU* activation functions.
- `FeaturePyramidNetwork`: This module's main goal is to address the challenge of detecting objects at different scales in an image. The pyramid consists of multiple feature maps at different scales. Each level of the pyramid represents a different scale of the original image, from fine details to high-level features.

- `RoIHeads`: Responsible for processing the regions of interest (RoIs) proposed by the detection module.
- `FastRCNNPredictor`: Module that predicts the class and bounding box regression for the *RoIs*.
- `Conv2d`: Convolutional layer from *PyTorch*.
- `LastLevelMaxPool`: Applied to the last level of the feature pyramid to reduce its spatial resolution. It is generally used to equalize the resolutions of the feature maps from different pyramid levels.

The model's structure follows a hierarchical approach, where the main network extracts the features from the input image, while the *Feature Pyramid Network (FPN)* creates multiple feature maps at different scales. These feature maps are essential for identifying regions and detecting objects in the later stages of the *Faster R-CNN* model.

Another main component of the model is the object tracker, which handles the assignment of detections to tracks, deciding whether to maintain, create, or remove tracked objects frame by frame. The file `tracker.py` defines the classes `Tracker`, responsible for tracking objects, and `Track`, which represents an individual tracking instance for a detected object. The `Tracker` class consists of the following methods:

- `__init__(self, obj_detect)`: Initializes the tracker with an object detection module.
- `reset(self, hard=True)`: Resets the tracker by clearing the tracks, track count, results, and image index.
- `add(self, new_boxes, new_scores)`: Initializes new tracking objects with the provided bounding boxes and scores, and stores them.
- `get_pos(self)`: Returns the positions of all active tracks.
- `data_association(self, boxes, score)`: Associates data with existing tracks and newly detected boxes according to the *IoU* criterion.
- `step(self, frame)`: Performs tracking for a single frame through object detection, data association, and result updating.
- `get_results(self)`: Returns the tracking results.

The `Track` class consists of the following attributes:

- `id`: Tracking ID.
- `box`: Bounding box of the detected object.
- `score`: Detector's confidence score for the detected object.

In the file where the model is called and other scripts are executed, a class called `TrackerIoUAssignment` is generated, which inherits from the `Tracker` class and

overrides the `data_association` function to include the *IoU* criterion that determines the best matches.

Another important file is `object_detector.py`. This class encapsulates a *Faster R-CNN* model with a *Feature Pyramid Network* (FPN) as its backbone, providing an interface for initializing the model with a specific number of classes and performing detections on input images. The `detect` function allows users to perform detections using this model and obtain the resulting bounding boxes and confidence scores. As its backbone, it uses a *ResNet-50* architecture without pre-trained weights, and the parameter `nms_thresh` indicates the threshold used to decide whether two detections should be merged.

There are also three additional files – `data_obj_detect.py`, `data_track.py`, and `utils.py` – which serve different purposes, but they are not discussed in detail since the focus of the model optimization lies mainly in the first two files.

A. High-Quality Detections

It has been shown that the performance of the object tracker can be improved in a very simple way by considering only high-quality detections from the detector. This is because low-quality detections, with very low scores, tend to be false positives or the result of noise from small or poorly illuminated objects that cause confusion. Filtering them reduces their number, which improves precision. It also enhances accuracy by prioritizing detections in which the model has higher confidence.

To achieve this, the documentation [3] associated with the *FasterRCNN* class was studied, which specifies four interesting parameters that can be fine-tuned:

- `box_score_thresh` (float): Minimum classification score threshold to define which detections are considered.
- `rpn_nms_thresh` (float): Threshold used to filter redundant responses with significant overlap between bounding boxes and the *RPN* (Region Proposal Network).
- `box_nms_thresh` (float): Threshold used to filter redundant responses with significant overlap between bounding boxes.
- `rpn_score_thresh`: RPN score threshold used to filter out less likely proposals before further processing.

These four parameters can be adjusted to improve object detection.

B. Efficient Data Association Algorithm

The association strategy employed by the `TrackerIoUAssignment` class can be viewed as a form of greedy search. In object tracking, greedy search involves making locally optimal decisions at each step without considering the global optimum.

To address this issue, the use of the Hungarian algorithm is proposed — a technique used to solve the assignment problem in object tracking. The Hungarian algorithm [2] iteratively searches for the optimal assignment by employing a

combination of matrix operations and augmenting paths. This ensures the globally optimal assignment with the minimum cost. In our context, in object tracking, we aim to use this technique to maximize the match between existing tracks and new detections, thus finding the assignment that maximizes the total gain.

A brief explanation of how this algorithm works in our context is as follows:

- **Distance matrix:** In object tracking, object detections in a frame can be represented as the rows of a matrix, and the tracking paths (or object detections from the previous frame) can be represented as the columns. Each cell in the matrix contains a similarity measure (e.g., *IoU* or Euclidean distance) between a detection and a track. This matrix is used as input to the Hungarian algorithm. In the context of the Hungarian algorithm, the distance matrix represents the cost of assigning each detection to each track. The objective is to minimize this cost, which is equivalent to maximizing the total similarity between detections and tracks. The Hungarian algorithm finds the optimal assignment by searching for augmenting paths in the distance matrix, ensuring that the globally optimal solution is found for the data association problem.
- **Cost matrix expansion:** The Hungarian algorithm assigns a single track to each detection and all detections. However, it might happen that leaving some previous detections unassigned yields a lower overall cost, meaning that the Hungarian algorithm alone cannot find the truly optimal solution considering this case. To solve this, the cost matrix is expanded to account for both missing and false detections, allowing for unassigned detections and tracks that disappear in the next frame. The only drawback of this approach is that we do not know the true costs of obtaining a false or missing detection, so these values are set empirically.
- **Application of the Hungarian algorithm:** Once the initial coverage has been established, the algorithm finds the best existing assignment while considering the possibility of false and missing detections.

III. IMPLEMENTATION

A. High-Confidence Detections

To include only high-confidence detections, certain given classes have been modified directly within the notebook where the entire practice is executed. It is considered that this increases the readability of the code, even though it decreases its robustness; however, we keep it this way since it would be difficult to encapsulate all these functions in the given `.py` files.

Thus, the following aspects of the *FRCNN-FPN* class [3] have been modified:

- `box_score_thresh`: a value equal to 0.81040990.
- `box_nms_thresh`: a value equal to 0.5.
- `rpn_nms_thresh`: a value equal to 0.7.
- `rpn_score_thresh`: a value equal to 0.0.

where the value of `box_score_thresh` was determined experimentally by performing parameter tuning on the *test* set, in order to obtain the highest possible *F1-score*, which combines precision and recall into a single metric. This parameter has a strong influence on model performance due to its role in filtering out low-confidence detections.

B. Efficient Data Association

To implement the Hungarian algorithm, the class `TrackerIoUAssignment.hungarian` was created, inheriting from the `Tracker` class—just like `TrackerIoUAssignment`—and modifying the `data_association` function:

- **Distance calculation based on *IoU*:** The distance between existing tracks and new detections is computed using Intersection over Union (*IoU*). A higher *IoU* indicates a better match between a detection and a track, meaning that a higher similarity is represented by a lower cost. For this part, the function `distances.iou_matrix` from the `motmetrics` library is used, with the parameter `max_iou` set to 0.5. This means that two bounding boxes are considered to have significant overlap if their intersection exceeds 50%. Finally, this matrix is extended to allow the application of the Hungarian algorithm with specific cost values.
- **Solving the association problem:** After calculating the costs between all possible combinations of detections and tracks, the `linear_sum_assignment` function from the `Scipy` library is used to solve the data association problem. This function applies the Hungarian algorithm to find the optimal assignment that minimizes total costs.
- **Obtaining Assignments:** Once the association problem has been solved, `linear_sum_assignment` returns two index arrays: `row_indices` and `col_indices`, which contain the indices of tracking tracks and object detections, respectively, that form the optimal assignments found by the Hungarian algorithm.

C. Handling Missing and False Detections

This modification was added to the `TrackerIoUAssignment.hungarian` class by implementing the `extend_cost_matrix` function and making the corresponding changes in the `data_association` method. The implementation of the cost matrix extension to handle false and missing detections—necessary for the Hungarian algorithm—is as follows:

- **Creation of the extended cost matrix:** The function `extend_cost_matrix` takes as input the cost matrix (*IoU*-based distances) and two thresholds: `mis_det_thresh`, which is used to fill the main diagonal of the extended cost matrix. Assigning a value different from 1 (in this case, `mis_det_thresh`) on the diagonal penalizes associations where detections

are not correctly matched with tracks from the previous frame, modeling missed detections. The second threshold, `false_det_thresh`, is used analogously to penalize false detections—cases where a new detection is incorrectly matched to a previous track. Finally, these cost matrices are concatenated to form the matrix used by the Hungarian algorithm.

- **Loop through assignment indices:** The algorithm iterates through the assignment indices obtained from the association matrix. These indices represent the associations between detections in the current frame and tracks from the previous frame.
- **Updating existing tracks:** If `i` and `j` are both smaller than `row_number` and `col_number`, respectively, it means there is a valid assignment between a previous and a current detection. In this case, the position and score of the corresponding track are updated with the new detection and score. `row_number` represents the number of detections in the current frame, and `col_number` represents the number of tracks detected in previous frames that are being considered for association.
- **Adding new tracks:** If `i` is smaller than `row_number` but `j` is greater than or equal to `col_number`, this indicates a false detection. We handle this by assuming it corresponds to a new object appearing in the image. Thus, the new detection and score are added to the lists `new_boxes` and `new_scores`.
- **Removing missing tracks:** If `i` is greater than or equal to `row_number` but `j` is smaller than `col_number`, this indicates a missing detection. We assume this means the object has left the image. The corresponding track ID is added to the `track_ids_to_remove` list.
- **Removing and adding tracks:** Once the loop is complete, tracks marked in `track_ids_to_remove` are deleted, and new tracks are added using the new detections and scores stored in `new_boxes` and `new_scores`.

D. Video Generation

Finally, the `export_to_mp4` function takes all the labeled images from each video and compiles them into a .mp4 video that shows the sequences with the overlaid tracking detections.

- 1) `plot_confusion_matrix(y_pred_proba, y_truth, threshold=0.5)`. Input parameters: `y_pred_proba` — the predicted probabilities from the model, which are converted into binary class labels using the specified threshold; `y_truth` — the ground-truth labels used to compute the confusion matrix; and `threshold`, which allows adjusting the model's sensitivity by changing the cutoff for classification. The function converts predicted probabilities into binary labels according to the threshold, computes the confusion matrix based on true and predicted labels, and visualizes the result using a color map.

- 2) `plot_precision_recall(y_pred_proba, y_truth)`. Input parameters: `y_pred_proba` — the predicted probabilities from the model, and `y_truth` — the true labels of the samples. This function visualizes model performance in terms of precision and recall, finds the optimal threshold based on the F1-score, and displays the confusion matrix for that optimal threshold.
- 3) `calcular_f1_max(y_truth, y_proba)`. Using the same input parameters as the previous methods, this function finds and returns the highest possible F1-score and the corresponding threshold, using the precision-recall curves.

E. Running the Code

To compile and run the code, simply execute the notebook cells sequentially as provided in the submission. The code was executed locally; therefore, the cells corresponding to downloading the base model have been commented out.

IV. EXPERIMENTS

A. Object Detector

This experiment aims to evaluate a previously trained object detection model, specifically a variant of the *Faster R-CNN* model with a feature pyramid network (*FRCNN_FPN*), using a test dataset. The evaluation process begins with loading the model and setting it up for inference. Once the model is ready, the test dataset is selected, which in this case consists of the sequences from the **MOT16** dataset.

The crucial step of the experiment involves evaluating the model on the test dataset. This includes generating model predictions on the test images and comparing these predictions with the *ground truth* annotations, which represent the actual locations of objects in the images. The comparison is carried out to compute important evaluation metrics such as *precision* and *recall*.

After computing these metrics, the experiment seeks to identify and optimize the appropriate threshold for the model's classification score. This is achieved by visualizing the *precision-recall* curve, which shows how *precision* and *recall* vary for different score thresholds. The ultimate goal is to select the threshold that simultaneously maximizes both *precision* and *recall*, indicating an optimal balance between the model's classification accuracy and detection capability.

At the end of the experiment, the optimal threshold is expected to have been identified, allowing the model to make accurate and comprehensive predictions about the presence and location of objects in the test images.

From this experiment, it was found that the optimal value for the `box_score_thresh` parameter is 0.81, which yields an improvement in the *F1-score* to a value of 95.57%.

B. Hungarian Algorithm

For the experiment to test the performance of the tracker using the Hungarian algorithm and the inclusion of the

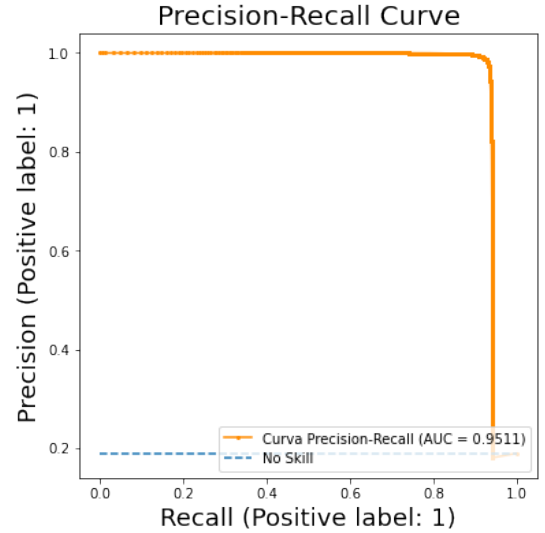


Fig. 1. Precision-recall curve for different `box_score_thresh` thresholds.

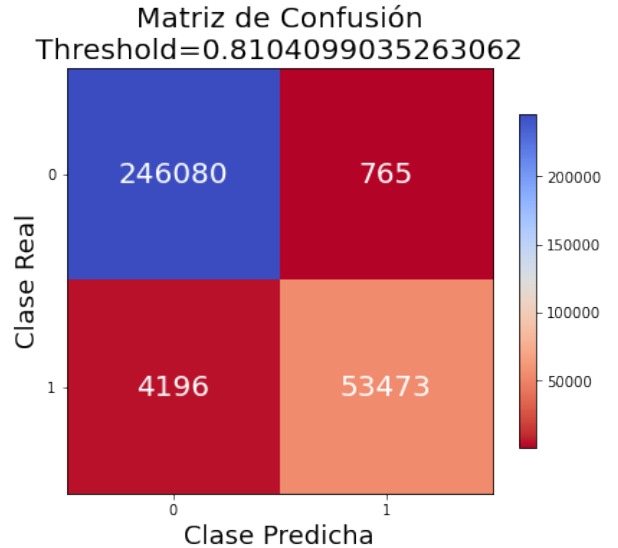


Fig. 2. Confusion matrix for the optimal `box_score_thresh` threshold of 0.81040990.

extended cost matrix, the tracker was executed and evaluated with the modified assignment algorithm, and the results were analyzed. This modification led to a significant improvement in detection assignment and test metrics.

V. DATA USED

The **MOT16** dataset [1] was used, which serves as a benchmark for evaluating multiple object tracking models.

It consists of annotated video sequences featuring multiple object tracking in realistic scenarios, both with static and moving cameras. The dataset is divided into 7 training sequences and 7 test sequences.

For training, the following subset of data was used:

- **MOT16-02**: Sequence captured around a square with people walking.

- **MOT16-04**: Sequence captured on a pedestrian street at night, from an elevated viewpoint.
- **MOT16-05**: Sequence captured on a street from a moving platform.
- **MOT16-09**: Sequence captured on a pedestrian street from a close-angle perspective.

For testing, the following subset of data was used:

- **MOT16-10**: Sequence captured on a pedestrian street at night with a moving camera.
- **MOT16-11**: Sequence captured with a forward-moving camera inside a busy shopping mall.
- **MOT16-13**: Sequence captured from a bus at a crowded intersection.

VI. RESULTS AND ANALYSIS

When comparing the evaluation of the baseline object detector with that of the detector using modified thresholds, a slight improvement can be observed in the metrics used by the evaluation method.

```
AP: 84.7773 %. Prec: 94.5962 %. Rec: 89.6079 %. TP: 51676. FP: 2952. F1-score: 92.0345 %.
CPU times: user 4min 57s, sys: 15.9 s, total: 5min 13s
Wall time: 5min 14s
```

Fig. 3. Baseline object detector.

```
AP: 91.4213 %. Prec: 98.5895 %. Rec: 92.7223 %. TP: 53472. FP: 765. F1-score: 95.5659 %.
CPU times: user 4min 59s, sys: 18.1 s, total: 5min 17s
Wall time: 5min 20s
```

Fig. 4. Object detector with fine-tuned `box_score_thresh`.

Regarding the improvement in the `tracker`, the process begins by running the baseline tracking algorithm using `IoU`. Then, the code is executed again using the algorithm with the implemented improvements explained previously, on the same dataset, in order to compare the results. The results obtained after applying the improved `tracker` with the adjusted thresholds are also included. The metrics returned by the `evaluate_mot_metrics` method, located in the `utils.py` file within the `trackers` directory, are compared.

```
Runtime for all sequences: 259.7 s.
IDF1 IDP IDR Rcll Prcn GT MT PT ML FP FN IDs FM MOTA MOTP
MOT16-13 46.5% 64.2% 36.5% 45.1% 79.2% 110 28 34 48 1375 6393 82 124 32.6% 0.134
MOT16-11 49.0% 54.1% 44.8% 55.8% 67.5% 75 15 32 28 2542 4166 20 39 28.7% 0.080
MOT16-10 35.3% 40.6% 31.3% 42.0% 54.5% 57 13 23 21 4504 7449 69 120 6.4% 0.138
OVERALL 42.9% 51.3% 36.8% 46.9% 65.4% 242 56 89 97 8421 18008 171 283 21.6% 0.118
```

Fig. 5. Baseline tracker.

It can be observed that there is a significant improvement compared to the baseline model across all metrics. Without increasing the computational cost, the addition of an optimal detection assignment algorithm and a threshold for detections in the object detector results in a notable enhancement. A smaller yet consistent improvement is also achieved by fine-tuning the thresholds of the *Faster-RCNN* model.

Visually, this can be observed by comparing frames obtained after executing each of the `trackers` separately.

```
Runtime for all sequences: 255.2 s.
IDF1 IDP IDR Rcll Prcn GT MT PT ML FP FN IDs FM MOTA MOTP
MOT16-13 48.5% 72.4% 36.4% 43.8% 87.0% 110 31 29 50 764 6548 71 76 36.6% 0.127
MOT16-11 52.5% 58.0% 48.0% 55.1% 66.7% 75 15 29 31 2600 4236 19 24 27.4% 0.075
MOT16-10 36.3% 46.5% 29.8% 40.9% 63.8% 57 13 22 22 2976 7590 54 83 17.3% 0.134
OVERALL 45.1% 57.5% 37.1% 45.8% 71.0% 242 59 80 103 6340 18374 144 183 26.7% 0.112
```

Fig. 6. Object tracker with fine-tuned `box_score_thresh` for the object detector.

```
Runtime for all sequences: 258.3 s.
IDF1 IDP IDR Rcll Prcn GT MT PT ML FP FN IDs FM MOTA MOTP
MOT16-13 66.6% 72.2% 61.9% 83.8% 97.7% 110 78 27 5 227 1887 321 182 79.1% 0.130
MOT16-11 66.7% 75.8% 59.7% 78.2% 99.3% 75 39 28 8 50 2055 55 48 77.1% 0.076
MOT16-10 52.2% 58.4% 47.1% 78.7% 97.6% 57 35 21 1 253 2737 266 222 74.6% 0.144
OVERALL 61.2% 68.0% 55.7% 80.3% 98.1% 242 152 76 14 530 6679 642 452 76.9% 0.121
```

Fig. 7. Object tracker with improved detector and Hungarian algorithm.



Fig. 8. Frame from **MOT16-10**. Baseline model.

Both algorithms perform well when the objects are clearly visible; however, when the objects are smaller or more difficult to distinguish, the improved model achieves higher precision than the baseline. It can be observed that one of the persons (ID-52 from **MOT16-10**, improved model) in the background is lost when using the baseline model's `tracker`, whereas the improved model successfully maintains tracking. Similarly, in the frames from **MOT16-13**, it is evident that in the baseline model, the bounding box with ID22 is not assigned to any person. To better appreciate the improvement, it is recommended to view the videos included in the submission under the `output` directory.

One observed weakness of the improved model is that, in moments with a considerable degree of occlusion, the `tracker` fails to correctly maintain tracking, causing the

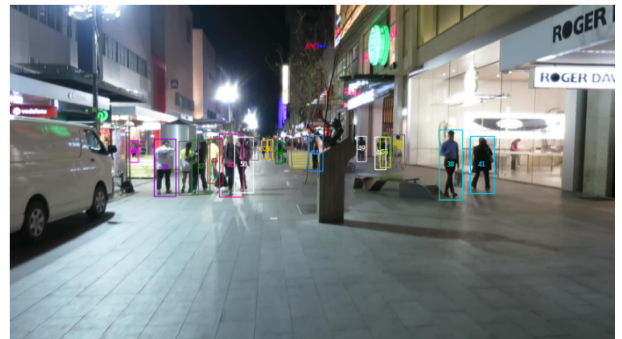


Fig. 9. Frame from **MOT16-10**. Final improved model.

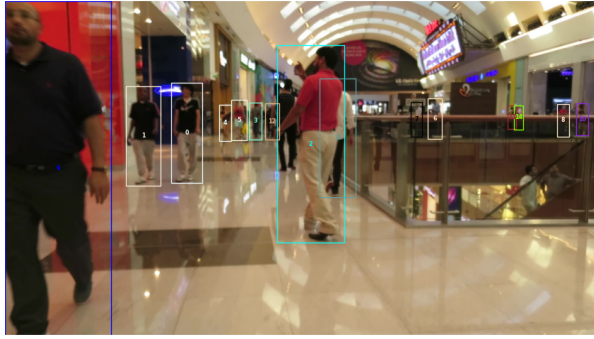


Fig. 10. Frame from **MOT16-11**. Baseline model.

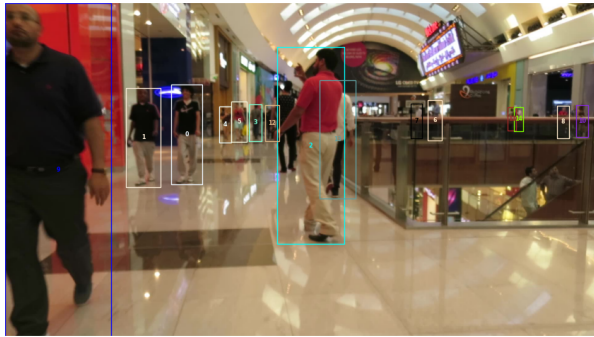


Fig. 11. Frame from **MOT16-11**. Final improved model.

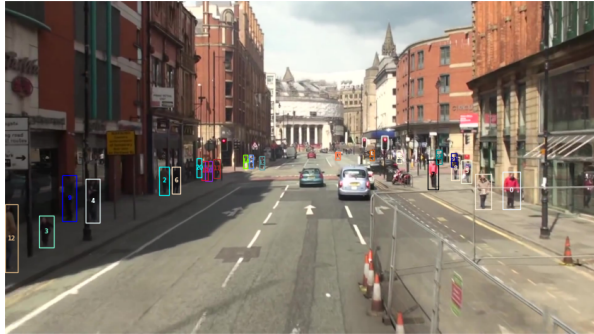


Fig. 12. Frame from **MOT16-13**. Baseline model.

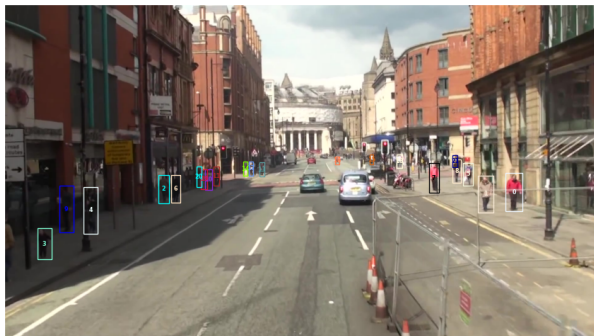


Fig. 13. Frame from **MOT16-13**. Final improved model.

object to disappear. Although the overall results and metrics are acceptable and significantly improved, there is still room for further enhancement, particularly in the IDF1, IDP, and IDR metrics. An interesting and shared weakness of both models is that, in the frames of **MOT16-11**, neither model is able to detect the two people ascending the stairs, who are clearly visible in the image—this is likely due to occlusion caused by the glass and its reflections.

VII. CONCLUSIONS

In view of the results, it can be concluded that the Hungarian algorithm demonstrates superior performance in assigning current detections to previous ones. As a future improvement, it is proposed to fine-tune all remaining parameters of the FasterRCNN class from the torchvision library.

As future work, to address the issue discussed in the results regarding the loss of tracking caused by object occlusion in the images, one possible approach would be to predict the trajectories of the objects. These predictions could even be combined with a *Kalman filter* [5] to reduce the uncertainty in the estimation.

VIII. TIME LOG

The approximate time spent on the different phases of the project has been as follows:

- Analysis and execution of the base model: 4 hours.
- Design of the improvement: 10 hours.
- Implementation and execution of the improvement: 13 hours.
- Writing of the report: 8 hours.

REFERENCES

- [1] <https://motchallenge.net/data/MOT16/>
- [2] <https://thinkautonomous.medium.com/computer-vision-for-tracking-8220759eee85>
- [3] https://github.com/pytorch/vision/blob/main/torchvision/models/detection/faster_rcnn.py
- [4] R.E. Burkard, M. Dell'Amico, S. Martello: *Assignment Problems* (Revised reprint). SIAM, Philadelphia (PA.) 2012.
- [5] Bewley, A., et al. *Simple online and realtime tracking*. IEEE ICIP 2016.